## OBJECT ORIENTED PROGRAMMING WITH C++

# ASSIGNMENT 1

### Design, Implement and Validate the Fare Logic of a City Metro Smart-Card System

| Submission detail | Information |
|---|---|
| Student name | Kaviya V |
| Register number | 192421022 |
| GitHub link | https://github.com/kaviyavellaisamy/DSA0107---C-/tree/main |
| Compiler/version | Record your installed MSYS2 GCC version |
| Submission date | 11/11/2026 |

Prepared from the supplied assignment brief. Fill in personal details, run the programs, record your actual compiler output, attach screenshots and commit each task separately. Do not present illustrative results as execution evidence.

## Scenario and assumptions

The metro has Inner (Rs 10), Middle (Rs 20) and Outer (Rs 30) zones. Peak periods are 08:00–10:00 and 17:00–20:00, with a 20% surcharge. A journey is allowed only if at least Rs 50 remains after deduction. The brief mentions student and senior-citizen concessions but does not state their percentage or calculation order; therefore the base code below does not invent a concession.

- Use start-inclusive, end-exclusive peak intervals: 08:00 and 17:00 are peak; 10:00 and 20:00 are off-peak.
- Store money as integer paise; 100 paise = Rs 1.
- A rejected journey never changes the card balance.
- Validate zone, hour, fare and recharge amount before processing.

## TASK 1 – Understand the Problem and Identify the Objects

| Object | Data | Behaviour / purpose |
|---|---|---|
| SmartCard | cardId, balancePaise | canTravel(), deduct(), recharge(); protects stored value and Rs 50 reserve. |
| Passenger | passengerId, name, category | Represents traveller and concession eligibility. |
| Journey | journeyId, origin, destination, zone, hour, fare, status | validate(), calculateFare(), complete(); records one trip. |
| Zone | named zone value | Restricts trips to valid fare categories. |
| FareTable | base fares, surcharge, minimum balance | baseFare(), calculateFare(); centralises policy. |

Procedural style uses separate variables and free functions such as calculateFare(balance, zone, hour). OOP groups related data and operations into classes, improving responsibility boundaries and maintainability.

- Encapsulation: SmartCard keeps balance private and changes it only through validated methods.
- Abstraction: the gate calls calculateFare() and canTravel() without knowing internal arithmetic.
- Inheritance: StudentPassenger and SeniorPassenger may derive from Passenger if their shared data and distinct concession rules justify it.
- Polymorphism: a virtual concession() function can be overridden for each passenger category once concession rules are defined.

Journey invariants: valid zone; hour 0–23; non-negative fare; balance after payment >= 5000 paise; rejected trips do not deduct. Procedural and OOP implementations both take O(1) time per journey and O(1) memory per card. OOP localises rule changes; procedural code may require edits in multiple functions.

## TASK 2 – Tokens, Keywords, Identifiers and Constants

- Keywords: int, bool, class, return, if, switch, constexpr.
- Identifiers: SmartCard, balancePaise, calculateFare, MIN_BALANCE_PAISE.
- Constants: 1000, 2000, 3000, 20, 5000.
- Operators: +, -, *, /, %, ==, >=, &&, ||, ?:.
- Punctuators: { }, ( ), ; , [ ], #.

```
constexpr long long INNER_FARE_PAISE = 1000;
constexpr long long MIDDLE_FARE_PAISE = 2000;
constexpr long long OUTER_FARE_PAISE = 3000;
```

```
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;
enum class Zone { Inner, Middle, Outer };
```

A macro (#define) is preprocessor text with no normal type or scope. const creates a read-only typed object. constexpr requires a compile-time constant expression and suits fixed fares. enum class provides scoped names and avoids implicit conversion to int. Use PascalCase for types, camelCase for variables/functions and UPPER_SNAKE_CASE for constants. Macro and constexpr usually need no runtime storage when compile-time substituted; execution time is effectively unchanged, but constexpr has type checking and scope.

## TASK 3 – Choose Data Types and Check Type Compatibility

| Value | Type | Reason |
|---|---|---|
| Fare and balance (paise) | long long | Exact integer arithmetic and large range. |
| Peak / validity | bool | True or false. |
| Zone | enum class Zone | Named, type-safe values. |
| Hour / minute | int | Small integer fields; validate ranges. |
| Journey count / total | long long | Millions of journeys and aggregate totals. |
| IDs | std::string | May contain letters or leading zeros. |
| Average | double | Suitable for statistics, not payable money. |

Money should be stored in paise, not double rupees, because binary floating point cannot exactly represent every decimal fraction. On common 64-bit GCC targets sizeof(bool)=1, char=1, int=4, long long=8, float=4 and double=8 bytes; verify locally because sizes are implementation-dependent.

```
#include <iostream>
int main() {
 std::cout << sizeof(bool) << ' ' << sizeof(char) << ' '
           << sizeof(int) << ' ' << sizeof(long long) << ' '
           << sizeof(float) << ' ' << sizeof(double) << '\\n';
}
```

Integer types promote in expressions; int combined with double converts the int to double. bool converts to 0 or 1 in arithmetic. Mixed signed/unsigned comparisons can convert a negative signed value to a large unsigned value. At the maximum peak fare Rs 36, 3,000,000 journeys total Rs 108,000,000 = 10,800,000,000 paise, beyond signed 32-bit int. Use long long.

Alternative double-rupee arithmetic and long-long-paise arithmetic are both O(1) per addition and commonly 8 bytes per value. The integer representation is exact to the paise; double can accumulate rounding error. Measure error for the actual test values rather than assuming a fixed amount.

## TASK 4 – Control Implicit Conversions

Conversions to identify: integer division before assignment to double; char promotion to int during arithmetic; int-to-bool conversion (zero=false, nonzero=true); signed-to-unsigned conversion in mixed comparisons.

### Predicted output

```
3 3.5
C 68
```

```
1 true
s >= u
```

Explanation: 7/2 is integer division and gives 3; static_cast<double>(7)/2 gives 3.5. 'A'+2 produces 'C'; ch+1 promotes char to int and prints 68. Integer 5 converts to true; default bool output is 1, then std::boolalpha prints true. Comparing -1 with unsigned 1 converts -1 to a large unsigned value, so the result is s >= u. The supplied program prints two spaces between 1 and true because the stream inserts a space before the boolalpha output.

Explicit casts and brace initialisation make conversions visible. Runtime and memory are generally unchanged; brace initialisation rejects narrowing at compile time.

```
int a{7}, b{2};
double c = static_cast<double>(a) / b;
bool flag = (5 != 0);
// int bad{3.7}; // compile-time narrowing error
```

# TASK 5 – Expressions and Operator Precedence

```
bool peak = (hour >= 8 && hour < 10) ||
            (hour >= 17 && hour < 20);
long long surcharge = peak ? base * 20 / 100 : 0;
long long finalFare = base + surcharge;
```

Relational operators bind before &&, and && before ||. Parentheses improve readability. Multiplication/division precede addition. x++ yields the old value; ++x increments before yielding. Avoid i=i++; use a separate increment. Integer division truncates toward zero: -10/3=-3 and -10%3=-1.

## Predicted output

```
11 16
9 6 4
6 4
3 1 -3 -1
18
```

Line 1: 5+3*2=11 and (5+3)*2=16. Line 2: x++ contributes old 5, ++y contributes new 4, so z=9; x=6 and y=4. Line 3: max is 6; (6%4)*2=4. Line 4: integer division/remainder results are 3, 1, -3, -1. Line 5: loop adds 0+3+6+9=18.

A bool peak[24] lookup is O(1), like the expression, but typically uses 24 bytes. It can reduce comparisons but must be initialised correctly.

# TASK 6 – Control Structures and Completed Code

Use switch to choose zone fare, if-else to validate balance, and ?: for a concise surcharge choice. Use range-based for for an array/vector of trips; for is suitable for a known count and while for sentinel-driven input. Use continue to skip an invalid trip and break to stop. Include default in switch.

## Pseudocode

```
READ zone, hour, balance
IF zone invalid OR hour outside 0..23: reject
peak = (hour >= 8 AND hour < 10) OR
       (hour >= 17 AND hour < 20)
SWITCH zone: Inner=1000, Middle=2000, Outer=3000
fare = base
IF peak: fare = base + base*20/100
IF balance-fare >= 5000: deduct and display
ELSE: deny; balance unchanged
```

### Completed C++ code

```
#include <iomanip>
#include <iostream>
#include <stdexcept>
using namespace std;
enum class Zone { Inner, Middle, Outer };
constexpr long long BASE_PAISE[] = {1000, 2000, 3000};
constexpr int PEAK_SURCHARGE_PERCENT = 20;
constexpr long long MIN_BALANCE_PAISE = 5000;
long long fare(Zone z, bool peak) {
 long long base = 0;
 switch (z) {
  case Zone::Inner: base=BASE_PAISE[0]; break;
  case Zone::Middle: base=BASE_PAISE[1]; break;
  case Zone::Outer: base=BASE_PAISE[2]; break;
  default: throw invalid_argument("Invalid zone");
 }
 return peak ? base + base*PEAK_SURCHARGE_PERCENT/100 : base;
}
bool canTravel(long long balance, long long f) {
 return f >= 0 && balance >= f && balance-f >= MIN_BALANCE_PAISE;
}
int main() {
 long long balance=10000;
 struct Trip { Zone zone; bool peak; };
 Trip trips[]={{Zone::Inner,false},{Zone::Outer,true},
               {Zone::Middle,false}};
 for (const Trip& t: trips) {
  long long f=fare(t.zone,t.peak);
  if (canTravel(balance,f)) {
   balance-=f;
   cout<<"Fare Rs "<<f/100<<'.'<<setw(2)<<setfill('0')
       <<f%100<<setfill(' ');
  } else cout<<"Denied (low balance)";
  cout<<" balance Rs "<<balance/100<<'\\n';
 }
}
```

Expected output: Fare Rs 10.00 balance Rs 90; Fare Rs 36.00 balance Rs 54; Denied (low balance) balance Rs 54. The last trip is denied because Rs 54 - Rs 20 = Rs 34, below the Rs 50 reserve. The 10:00 and 20:00 boundaries are off-peak under the half-open intervals [08:00,10:00) and [17:00,20:00).

An indexed fare array and switch are both O(1). The array uses storage for three fare values and is easy to extend, but index validation is essential. A switch may compile to a jump table.

## TASK 7 – Procedural and Object-Oriented Versions

```
// Procedural
bool canTravel(long long balance,long long f) {
 return balance>=f && balance-f>=5000;
}
bool deduct(long long& balance,long long f) {
 if (!canTravel(balance,f)) return false;
 balance-=f; return true;
}
// Object-oriented
class SmartCard {
 long long balancePaise; // private
public:
 explicit SmartCard(long long initial):balancePaise(initial){}
 long long balance() const { return balancePaise; }
```

```
 bool canTravel(long long f) const {
  return f>=0 && balancePaise>=f && balancePaise-f>=5000;
 }
 bool deduct(long long f) {
  if (!canTravel(f)) return false;
  balancePaise-=f; return true;
 }
};
```

Both versions are O(1) per check/deduction and O(1) storage per card. A private balance prevents outside code from bypassing validation. If it were public, any part of the program could set a negative balance or leave less than Rs 50. Centralising the minimum-balance constant simplifies rule changes.

## TASK 8 – Safe Input and Output

```
#include <iostream>
#include <limits>
int main() {
 int hour;
 std::cout << "Enter hour (0-23): ";
 while (!(std::cin >> hour) || hour<0 || hour>23) {
  std::cout << "Invalid hour. Enter 0-23: ";
  std::cin.clear();
  std::cin.ignore(std::numeric_limits<std::streamsize>::max(),'\\n');
 }
 std::cout << "Accepted hour: " << hour << '\\n';
}
```

The failed stream condition detects letters entered instead of a number. cin.clear() resets error flags; cin.ignore() discards the invalid line. Use fixed and setprecision(2) for decimal output, or divide integer paise by 100 and print the remainder using setw(2), setfill('0'). getline()+stoi() with try/catch processes a line in O(L) time and uses O(L) buffer space; it gives explicit control over malformed lines.

## TASK 9 – Implementation and Testing

| Test case | Input | Expected result | Actual result to fill |
|---|---|---|---|
| Inner off-peak | Inner, 11:00 | Rs 10.00 | _____ |
| Middle off-peak | Middle, 12:00 | Rs 20.00 | _____ |
| Outer off-peak | Outer, 12:00 | Rs 30.00 | _____ |
| Inner peak | Inner, 08:00 | Rs 12.00 | _____ |
| Outer peak | Outer, 17:00 | Rs 36.00 | _____ |
| 10:00 boundary | Inner, 10:00 | Rs 10.00 | _____ |
| 20:00 boundary | Inner, 20:00 | Rs 10.00 | _____ |
| Exact reserve | Balance Rs 60; fare Rs 10 | Allowed; Rs 50 remains | _____ |
| Below reserve | Balance Rs 59.99; fare Rs 10 | Denied; unchanged | _____ |
| Invalid zone | Unknown zone | Rejected | _____ |
| Negative recharge | -Rs 5 | Rejected; unchanged | _____ |
| Hour as letters | abc | Error handled and retried | _____ |

Five-trip manual example from Rs 200: Rs 10 (Inner off-peak) + Rs 24 (Middle peak) + Rs 30 (Outer off-peak) + Rs 12 (Inner peak) + Rs 36 (Outer peak) = Rs 112. Ending balance = Rs 88. Each trip leaves at least Rs 50, so all are allowed.

A table-driven test loop takes O(t) time for t tests and O(t) storage. A new boundary case is added as another row.

# TASK 10 – Expectations and Execution

The brief requires measured totals. The following are theoretical totals for a constant Rs 10 fare; run your own program and record the actual measurements.

| Journeys | Total rupees | Total paise | Fits signed 32-bit int paise? |
|---|---|---|---|
| 20,000 | 200,000 | 20,000,000 | Yes |
| 500,000 | 5,000,000 | 500,000,000 | Yes |
| 3,000,000 | 30,000,000 | 3,000,000,000 | No |

At Rs 36 maximum fare, 3,000,000 trips total Rs 108,000,000 = 10,800,000,000 paise. This exceeds signed 32-bit int. long long is appropriate. double has a large range but is not the preferred exact currency representation. Each addition is O(1); typical int size is 4 bytes and long long/double 8 bytes. Compile with -Wall -Wextra and document the actual warnings; do not invent warning output.

# TASK 11 – Recommend and Defend the Final Design

- Use enum class Zone and constexpr fare/surcharge/minimum-balance constants.
- Store money in long long paise; use long long for network totals.
- Use SmartCard with private balance and validated deduct/recharge methods.
- Use Journey for trip records and FareTable for central fare calculation.
- Validate input and boundary times; keep regression tests for normal, peak, invalid and exact-reserve cases.

A fare revision should update the central fare table and rerun regression tests. Limitations: concession percentages and their interaction with peak surcharge are unspecified. A configuration file makes fare changes possible without recompilation, but adds O(z) start-up parsing and O(z) storage for z zone fares, plus file validation. Compiled constexpr values are simpler and compile-time checked.

# TASK 12 – Design Challenge: Fourth Zone, Weekend Discount and Monthly Pass

New rules: Airport zone Rs 60; weekend discount 10% which does not combine with peak surcharge; monthly pass gives unlimited journeys in two chosen zones.

## A. Impact analysis

- Add Zone::Airport and 6000 paise to the fare table.
- Update zone validation, fare table size and any switch statements.
- Add weekend/date input and a separate discount branch.
- Add a Pass class storing two allowed zones, validity dates and active status.
- Record whether each journey used a pass or paid fare.

## B. Evaluate original design

The switch can be extended with an Airport case, but all zone switches need updating. A checked indexed table reduces duplicated fare definitions. Keep SmartCard for pay-as-you-go balance; monthly pass eligibility is a separate responsibility and belongs in Pass.

Worked examples: Airport weekday off-peak Rs 60; Airport weekday peak Rs 72; Airport weekend Rs 54. The weekend discount replaces the peak surcharge, even if the journey occurs during peak time.

## C. Revised design code

```
#include <array>
#include <stdexcept>
enum class Zone { Inner, Middle, Outer, Airport };
constexpr std::array<long long,4> BASE_FARE={1000,2000,3000,6000};
long long revisedFare(Zone z,bool peak,bool weekend) {
 int i=static_cast<int>(z);
 if(i<0 || i>=static_cast<int>(BASE_FARE.size()))
  throw std::invalid_argument("Invalid zone");
 long long base=BASE_FARE[static_cast<std::size_t>(i)];
 if(weekend) return base-(base*10/100); // no peak surcharge
 if(peak) return base+(base*20/100);
 return base;
}
class Pass {
 Zone zone1,zone2; bool active;
public:
 Pass(Zone a,Zone b):zone1(a),zone2(b),active(true){}
 bool allows(Zone z) const {
  return active && (z==zone1 || z==zone2);
 }
 void deactivate(){active=false;}
};
```

A complete Pass should also store holder/card identity, start date and expiry date, and reject duplicate selected zones. If an active pass covers the journey zone, authorise without deducting pay-as-you-go fare; otherwise calculate and deduct the normal fare.

## D. Evidence-based defence

Recommendation: MODIFY, not fully redesign. Retain SmartCard encapsulation and Journey records; replace the three-zone switch with a four-entry checked fare table; add a weekend rule and separate Pass class. This contains change while separating stored-value payment from subscription entitlement.

| Scenario | Expected result |
|---|---|
| Airport weekday off-peak | Rs 60.00 |
| Airport weekday peak | Rs 72.00 |
| Airport weekend, including peak time | Rs 54.00 |
| Active pass, selected zone | No fare deduction |
| Active pass, unselected zone | Normal fare |
| Expired pass | Normal fare |

SAMPLE SIMULATION OUTPUT — METRO SMART-CARD FARE SYSTEM

```
PS C:\MetroSmartCard> g++ -std=c++17 metro_fare.cpp -o metro_fare.exe
PS C:\MetroSmartCard> .\metro_fare.exe

========== METRO SMART-CARD FARE SIMULATION ==========
Initial card balance : Rs 200.00
Minimum balance rule : Rs 50.00 must remain after payment
Peak surcharge       : 20%  (08:00-10:00, 17:00-20:00)

JOURNEY TESTS
------------------------------------------------------
1. Inner Zone   | 11:00 | Off-peak | Fare Rs 10.00 | APPROVED
   Remaining balance: Rs 190.00
2. Middle Zone  | 08:30 | Peak     | Fare Rs 24.00 | APPROVED
   Remaining balance: Rs 166.00
3. Outer Zone   | 12:00 | Off-peak | Fare Rs 30.00 | APPROVED
   Remaining balance: Rs 136.00
4. Inner Zone   | 17:30 | Peak     | Fare Rs 12.00 | APPROVED
   Remaining balance: Rs 124.00
5. Outer Zone   | 18:00 | Peak     | Fare Rs 36.00 | APPROVED
   Remaining balance: Rs 88.00

VALIDATION TESTS
------------------------------------------------------
6. Invalid zone  | REJECTED — invalid zone selection
7. Invalid hour  | REJECTED — hour must be 0 to 23
8. Low balance   | DENIED — Rs 50.00 minimum not preserved
   Balance unchanged after rejected journey.

SIMULATION SUMMARY
------------------------------------------------------
```

## Submission checklist

- Create repository DSA01-<RegisterNumber>-Assignment1 and share with faculty.
- Create Task01 through Task12 folders with source, input and output screenshots.
- README.md must contain name, register number, compiler/version, exact compile commands, run steps and result notes.
- Compile with: g++ -std=c++17 -Wall -Wextra filename.cpp -o filename.exe
- Commit progressively, at least one commit per task; include working GitHub link in the report.

Important: this report is a model answer based on the supplied assignment. The assignment requires your own execution, actual compiler/version, screenshots, measured results, and progressive GitHub commits. Complete those before submission and ensure you can explain the code in the verification discussion.